Advanced Data Structures & Algorithm Analysis (2501CS10)

Unit-wise Important Questions — Answers | Aditya University

All worked problems below use the data given in the figures/tables of your question paper. Answers were verified by running the algorithms.

UNIT I — Algorithm Analysis and Hashing

Q1. Explain the importance of algorithm analysis. Differentiate between time complexity and space complexity with suitable examples.

Algorithm analysis is the process of estimating the resources (time and memory) an algorithm needs as a function of the input size *n*, independent of machine and language.

Importance:

1. Compare different algorithms for the same problem and choose the best one.
2. Predict performance for large inputs before implementation.
3. Identify the best, average and worst case behaviour.
4. Avoid wasting time and memory (scalability).
5. Gives a machine-independent measure using asymptotic notation.

Aspect	Time Complexity	Space Complexity
Meaning	Amount of time (number of basic operations) as a function of n	Amount of memory needed as a function of n
Measured by	Counting steps/operations	Input space + auxiliary (extra) space + recursion stack
Unit	Number of operations	Number of memory words/bytes
Example 1	Linear search: O(n)	Linear search: O(1) extra space
Example 2	Merge sort: O(n log n)	Merge sort: O(n) auxiliary array
Example 3	Matrix addition: $O(n^2)$	Matrix addition: $O(n^2)$ for result
Trade-off	Can often be reduced by using more memory (e.g. hashing, DP tables)	Can be reduced by recomputation, usually at the cost of time

Q2. Describe the asymptotic notations O, Ω and Θ. Illustrate each notation with suitable examples.

Asymptotic notations describe the growth rate of a function f(n) for large n.

1. Big-O (upper bound, worst case)

f(n) = O(g(n)) if there exist constants c > 0 and n_0 such that **$f(n) \le c \cdot g(n)$ for all $n \ge n_0$**.

Example: f(n) = 3n + 2. Since 3n + 2 ≤ 4n for n ≥ 2, f(n) = O(n) (c = 4, n_0 = 2).

2. Big-Omega Ω (lower bound, best case)

f(n) = Ω(g(n)) if there exist c > 0 and n_0 such that **$f(n) \ge c \cdot g(n)$ for all $n \ge n_0$**.

Example: 3n + 2 ≥ 3n for n ≥ 1, so f(n) = Ω(n) (c = 3, n_0 = 1).

3. Big-Theta Θ (tight bound)

f(n) = Θ(g(n)) if there exist c_1, c_2 > 0 and n_0 such that **$c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_0$**.

Example: 3n ≤ 3n + 2 ≤ 4n for n ≥ 2, so f(n) = Θ(n). Equivalent to f(n) = O(g(n)) **and** f(n) = Ω(g(n)).

Notation	Meaning	Bound	Typical use
O(g)	f grows no faster than g	Upper	Worst case
Ω(g)	f grows at least as fast as g	Lower	Best case
Θ(g)	f grows exactly like g	Tight	Exact rate

Order of growth: $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$

Q3. Analyze the time complexity of the matrix multiplication algorithm (tabular method) using asymptotic notation and justify your answer.

Algorithm (for two n x n matrices A and B):

```
for i = 1 to n                      // runs n times
    for j = 1 to n                  // runs n times for each i
        C[i][j] = 0
        for k = 1 to n              // runs n times for each (i, j)
            C[i][j] = C[i][j] + A[i][k] * B[k][j]
```

Analysis:

- The innermost statement executes once for every combination of (i, j, k): **n x n x n = n^3** times.
- The statement `C[i][j] = 0` executes n^2 times.
- Loop overhead is proportional to n^3 as well.

$T(n) = c_1 \cdot n^3 + c_2 \cdot n^2 + c_3 \cdot n + c_4$

Justification:

- Upper bound: $T(n) \leq c \cdot n^3$ for large n, so **$T(n) = O(n^3)$**.
- Lower bound: $T(n) \geq c' \cdot n^3$ (at least n^3 multiplications are performed in every case), so **$T(n) = \Omega(n^3)$**.
- Therefore **$T(n) = \Theta(n^3)$** in best, average and worst case (the running time does not depend on the values in the matrices).

Space complexity: $\Theta(n^2)$ for the result matrix. *(Strassen's method reduces the time to $O(n^{2.81})$.)*

Q4. Explain the concept of hashing. Discuss different hash functions with suitable examples.

Hashing is a technique to store and retrieve data in nearly **O(1)** time. A **hash function** h(k) maps a key k to an index (address) in a **hash table** of size m. Insert, search and delete use this index directly.

```
key  --->  h(key)  --->  index in hash table T[0 .. m-1]
```

Characteristics of a good hash function: easy and fast to compute; distributes keys uniformly; minimises collisions; uses all parts of the key.

Common hash functions:

1. Division method: $h(k) = k \bmod m$ (m is preferably a prime). *Example:* m = 10, k = 237, so h = 237 mod 10 = 7.

2. Multiplication method: $h(k) = \text{floor}(m \cdot (k \cdot A \bmod 1))$, where $0 < A < 1$ (e.g. $A \approx 0.618$). *Example:* k = 123, m = 100, A = 0.618: $k \cdot A = 76.014$, fractional part 0.014, so h = floor(100 x 0.014) = 1.

3. Mid-square method: Square the key and take the middle digits. *Example:* k = 123, $k^2 = 15129$, middle digits "12" gives h = 12.

4. Folding method: Divide the key into equal parts and add them (shift or boundary folding). *Example:* k = 123456, parts 12 + 34 + 56 = 102, so h = 102 mod 100 = 2.

5. Digit analysis: Choose the digits of the key that are most uniformly distributed (used when all keys are known in advance).

Q5. Explain the collision resolution techniques: Linear Probing, Quadratic Probing, Double Hashing and Separate Chaining.

A **collision** occurs when $h(k_1) = h(k_2)$ for two different keys. Techniques are of two kinds: **open addressing** (probing) and **chaining**.

Example data: table size m = 10, keys **23, 43, 13, 27**, h(k) = k mod 10. Keys 23, 43, 13 all hash to index 3.

1. Linear Probing: h(k, i) = (h(k) + i) mod m. Try the next free slot sequentially.

- 23 goes to 3; 43 to 3 (occupied), try 4, so 4; 13 to 3, 4 occupied, so 5; 27 goes to 7.
- Table: 3:23, 4:43, 5:13, 7:27.
- *Drawback:* **primary clustering**.

2. Quadratic Probing: h(k, i) = (h(k) + i^2) mod m.

- 23 to 3; 43 to 3 (occupied), 3+1 = 4, so 4; 13 to 3, 4 occupied, 3+4 = 7, so 7; 27 to 7 (occupied), 7+1 = 8, so 8.
- Table: 3:23, 4:43, 7:13, 8:27.
- Reduces primary clustering but causes **secondary clustering**.

3. Double Hashing: h(k, i) = (h_1(k) + i·h_2(k)) mod m, with h_2(k) = 7 − (k mod 7).

- 23 to 3; 43: h_2 = 7 − 1 = 6, (3+6) mod 10 = 9, so 9; 13: h_2 = 7 − 6 = 1, (3+1) = 4, so 4; 27 goes to 7.
- Table: 3:23, 4:13, 7:27, 9:43.
- Best distribution; avoids both primary and secondary clustering.

4. Separate Chaining: Each table slot holds a linked list of all keys that hash to it.

- Slot 3: 23 → 43 → 13; slot 7: 27.
- Table never overflows; load factor can exceed 1; needs extra memory for pointers.

Method	Clustering	Extra memory	Table can overflow
Linear probing	Primary	No	Yes
Quadratic probing	Secondary	No	Yes
Double hashing	Minimal	No	Yes
Separate chaining	None	Yes (pointers)	No

Q6. Differentiate between Static Hashing and Dynamic Hashing. Explain the working of Dynamic (Extendable) Hashing with an example.

Feature	Static Hashing	Dynamic Hashing
Table size	Fixed number of buckets	Grows and shrinks with the data
Hash function	Fixed, h(k) = k mod m	Uses a variable number of bits of the hash value
Overflow	Needs overflow chains/probing; performance degrades	Handled by splitting buckets
Reorganisation	Costly full rehashing when the table is full	Only one bucket is split at a time
Space	Wasted if the file is small, overflow if large	Efficient use of space
Example	Simple hash table	Extendable hashing, linear hashing

Extendable Hashing — working:

- A **directory** of 2^d pointers is indexed by the first/last *d* bits of h(k) (d = **global depth**).
- Each **bucket** has a **local depth** d' ≤ d and a fixed capacity.
- Several directory entries may point to the same bucket.
- **Insert:** find the bucket using d bits. If there is space, insert. If the bucket is full:
  - If **d' < d**: split the bucket, increase its local depth, redistribute keys.
  - If **d' = d**: **double the directory** (d = d + 1), then split the bucket.

Example: bucket capacity = 2, using the **last bits** of the binary key. Initial global depth d = 1.

1. Insert 4 (100) goes to bucket "0": {4}. Insert 1 (001) goes to bucket "1": {1}. Insert 5 (101) goes to bucket "1": {1, 5} (now full).
2. Insert 7 (111): bucket "1" is full and d' = d = 1, so **double the directory** (d = 2) and split the bucket using 2 bits.
3. Keys with last two bits **01** are 1 and 5; keys with **11** is 7.

Directory (last 2 bits)	Points to bucket	Local depth
00	B0 = {4}	1
10	B0 = {4} (shared)	1
01	B1 = {1, 5}	2
11	B2 = {7}	2

Advantages: no full rehash, grows gracefully, at most two disk accesses. **Disadvantage:** the directory itself can become large.

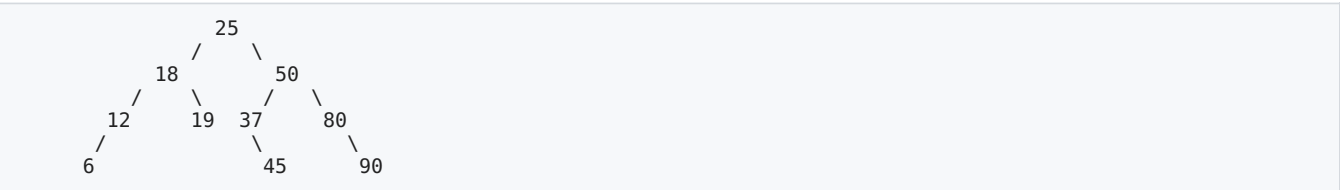
UNIT II — Balanced Trees and Heaps

Q1. Construct an AVL tree by inserting the keys 50, 25, 12, 80, 90, 37, 45, 18, 19, 6. Show the required rotations.

An AVL tree keeps the **balance factor** (height of left subtree − height of right subtree) of every node in {−1, 0, +1}.

Step	Insert	Result / problem	Rotation
1	50	50	none
2	25	50 (left 25)	none
3	12	12 under 25 under 50: node 50 has balance +2 (LL case)	**Right rotation at 50** → 25 (12, 50)
4	80	80 right of 50	none
5	90	90 right of 80: node 50 has balance −2 (RR case)	**Left rotation at 50** → 25 (12, 80 (50, 90))
6	37	37 left of 50: node 25 has balance −2, right child 80 is left-heavy (RL case)	**Right rotation at 80, then left rotation at 25** → 50 (25 (12, 37), 80 (–, 90))
7	45	45 right of 37	none (all balanced)
8	18	18 right of 12	none
9	19	19 right of 18: node 12 has balance −2 (RR case)	**Left rotation at 12** → 18 (12, 19)
10	6	6 left of 12: node 50 has balance +2, left child 25 is left-heavy (LL case)	**Right rotation at 50**

Final AVL tree:



Height = 4, all balance factors are in {−1, 0, +1}. Rotations used: 2 single-right (LL), 2 single-left (RR), 1 double (RL).

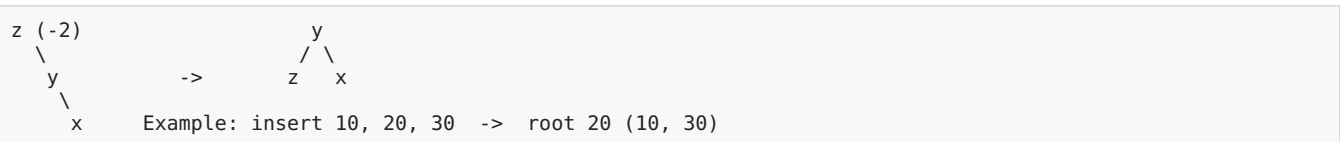
Q2. Explain the four types of AVL tree rotations with suitable examples. Discuss the applications of AVL trees.

Let **z** be the lowest unbalanced node, **y** its taller child, and **x** the taller child of y.

1. LL Rotation (Right rotation) — insertion into left subtree of left child.



2. RR Rotation (Left rotation) — insertion into right subtree of right child.



3. LR Rotation (Left-Right) — insertion into right subtree of left child. Left rotation at y, then right rotation at z.

```
    z                z              x
   /                /              / \
  y        ->      x      ->      y   z
   \              /
    x            y       Example: insert 30, 10, 20  ->  root 20 (10, 30)
```

4. RL Rotation (Right–Left) — insertion into left subtree of right child. Right rotation at y, then left rotation at z.

```
z                  z                  x
  \                  \                / \
   y       ->         x     ->       z   y
  /                    \
 x                      y   Example: insert 10, 30, 20  ->  root 20 (10, 30)
```

Case	Imbalance	Fix
LL	Left-Left	Single right rotation
RR	Right-Right	Single left rotation
LR	Left-Right	Left then right (double)
RL	Right-Left	Right then left (double)

Applications of AVL trees:

- Databases and indexing where lookups dominate insertions.
- In-memory dictionaries and sets requiring guaranteed O(log n) search.
- Lookup tables and routing tables; memory management.
- Applications that need ordered data with fast search (e.g. spell-checkers, file systems directories).

Search, insert and delete are all **O(log n)** worst case.

Q3. Explain the deletion procedure in an AVL tree. Show the balancing operations after deletion of 15 from the given AVL tree.

Deletion procedure:

1. Delete the node as in a BST (leaf: remove; one child: replace by child; two children: replace by inorder successor/predecessor and delete that node).
2. Walk back up to the root, updating heights and balance factors.
3. At the first unbalanced node, perform the suitable rotation (LL, RR, LR, RL). For deletion, the rotation type is chosen by the balance factor of the **taller child** (if it is 0, a single rotation is used).
4. **Continue upward**, because a rotation may reduce the height and cause further imbalance (unlike insertion, more than one rotation may be needed).

Given tree: root 50; 50 (25 (15, 40 (35, –)), 75 (60 (55, 65 (62, –)), 80 (–, 90))).

Delete 15 (a leaf):

1. Remove 15. Node **25** now has no left child and right child 40 (height 2): balance factor = −2.
2. The right child 40 is left-heavy (it has left child 35), so this is the **RL case**. Right rotate at 40, then left rotate at 25. Result: **35 (25, 40)**.
3. Moving up, the subtree at 35 now has height 2 while the right subtree of 50 (rooted at 75) has height 4: node **50** has balance factor = −2.
4. The right child 75 is left-heavy (60 side is taller), so again an **RL case**: right rotate at 75, then left rotate at 50. Node 60 becomes the new root.

Final tree after deletion of 15:

```
                60
              /    \
           50        75
          /  \      /  \
        35    55  65    80
       /  \      /       \
     25    40  62         90
```

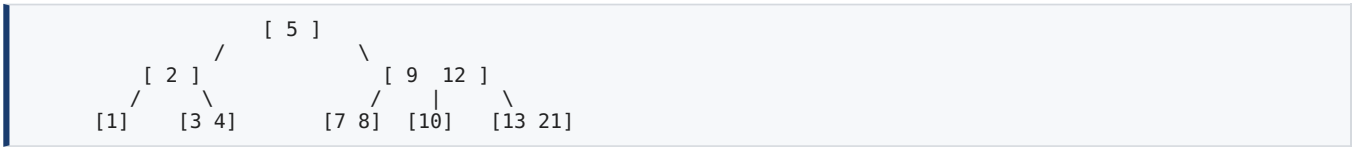
All balance factors are within {−1, 0, +1}. Two double (RL) rotations were required.

Q4. Construct a B-Tree of order 4 by inserting the keys 5, 3, 21, 9, 1, 13, 2, 7, 10, 12, 4, 8.

B-tree of order 4: each node has at most **4 children (3 keys)** and at least 2 children (1 key) except the root; all leaves are at the same level. When a node overflows (4 keys), it is split and the **2nd key moves up** to the parent.

Insert	Tree after insertion
5, 3, 21	[3 5 21]
9	Overflow [3 5 9 21], split, 5 goes up: **[5]** with children [3] and [9 21]
1	[5] → [1 3], [9 21]
13	[5] → [1 3], [9 13 21]
2	[5] → [1 2 3], [9 13 21]
7	[7 9 13 21] overflows, 9 goes up: **[5 9]** → [1 2 3], [7], [13 21]
10	[5 9] → [1 2 3], [7], [10 13 21]
12	[10 12 13 21] overflows, 12 goes up: **[5 9 12]** → [1 2 3], [7], [10], [13 21]
4	[1 2 3 4] overflows, 2 goes up; the parent [2 5 9 12] overflows too, 5 goes up: new root **[5]**
8	goes to leaf [7] giving [7 8]

Final B-tree:



Height = 3 (root + 2 more levels). *(If the 3rd key is chosen as the median for splitting, the shape differs slightly; always state the convention you use.)*

Q5. Explain the deletion operation in a B-Tree with a suitable example, and the applications of B-Trees.

Deletion cases (order m, minimum keys = ⌈m/2⌉ − 1; for order 4 the minimum is 1 key):

1. **Key in a leaf with more than the minimum keys:** simply remove it.
2. **Key in a leaf with minimum keys (underflow):**
   - **Borrow** from a sibling that has extra keys (rotate through the parent), or
   - **Merge** with a sibling and the separating key from the parent if no sibling can lend.
3. **Key in an internal node:** replace it with its **inorder predecessor or successor** (from a leaf) and delete that key from the leaf (then apply case 1 or 2).
4. If merging makes the parent underflow, repeat upward; if the root becomes empty, the tree height shrinks by one.

Example (using the tree built in Q4): root [5]; internal nodes [2] and [9 12]; leaves [1], [3 4], [7 8], [10], [13 21].

- **Delete 4:** leaf [3 4] has 2 keys, so simply remove 4, giving [3].
- **Delete 10:** leaf [10] would become empty (underflow). Its left sibling [7 8] has a spare key, so **borrow through the parent**: 9 moves down into the leaf and 8 moves up. Result: parent [8 12], leaves [7], [9], [13 21].
- **Delete 1:** leaf [1] becomes empty and its sibling [3] has no spare key, so **merge** the empty leaf, the separator 2 and [3] into the leaf [2 3]. The parent [2] is now empty (underflow). Its sibling [8 12] has a spare key, so borrow through the root: 5 moves down, 8 moves up, and leaf [7] becomes the child of the node holding 5.

Final tree after the three deletions:

```
              [ 8 ]
            /       \
        [ 5 ]         [ 12 ]
       /     \        /      \
   [2 3]    [7]     [9]    [13 21]
```

Applications of B-trees:

- Database indexes (MySQL InnoDB uses B+-trees).
- File systems (NTFS, ext4, HFS+, Btrfs) for directory and metadata indexing.
- Storing large data on disk, since a node corresponds to a disk block, so tree height (disk accesses) is minimal.
- Multilevel indexing, key-value stores and search engines.

Search, insert, delete: **O(log n)**.

Q6. Construct a Min/Max Heap by inserting the elements 40, 15, 30, 10, 12, 6, 25, 18. Show all intermediate steps.

Max Heap (parent ≥ children): insert at the end, then **sift up** while larger than the parent.

Insert	Array representation after insertion	Action
40	[40]	root
15	[40, 15]	15 < 40, stays
30	[40, 15, 30]	30 < 40, stays
10	[40, 15, 30, 10]	10 < 15, stays
12	[40, 15, 30, 10, 12]	12 < 15, stays
6	[40, 15, 30, 10, 12, 6]	6 < 30, stays
25	[40, 15, 30, 10, 12, 6, 25]	25 < 30, stays
18	[40, 18, 30, 15, 12, 6, 25, 10]	18 > 10 swap; 18 > 15 swap; 18 < 40 stop

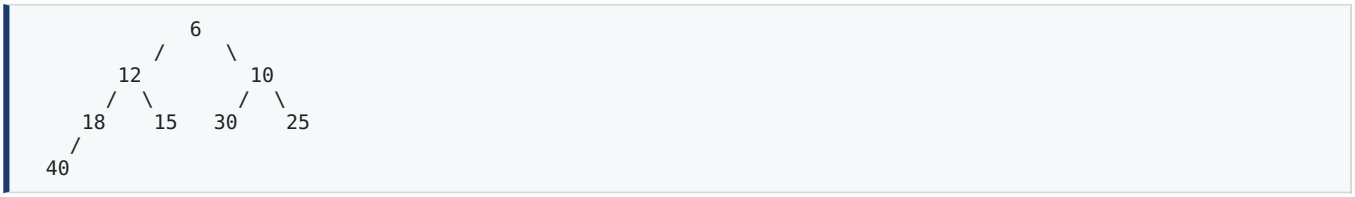
Final Max Heap:

```
            40
          /    \
        18       30
       /  \     /  \
     15   12   6    25
    /
  10
```

Min Heap (parent ≤ children): sift up while smaller than the parent.

Insert	Array after insertion	Action
40	[40]	root
15	[15, 40]	15 < 40 swap
30	[15, 40, 30]	30 > 15, stays
10	[10, 15, 30, 40]	10 < 40 swap; 10 < 15 swap (becomes root)
12	[10, 12, 30, 40, 15]	12 < 15 swap; 12 > 10 stop
6	[6, 12, 10, 40, 15, 30]	6 < 30 swap; 6 < 10 swap; root
25	[6, 12, 10, 40, 15, 30, 25]	25 > 10, stays
18	[6, 12, 10, 18, 15, 30, 25, 40]	18 < 40 swap; 18 > 12 stop

Final Min Heap:



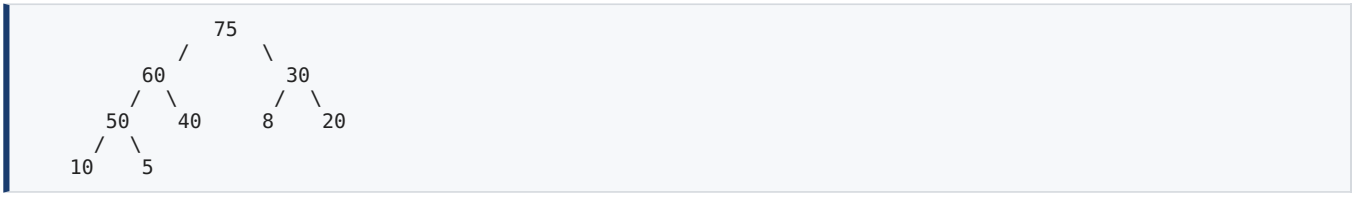
Q7. Perform one deletion operation on the given Max Heap and show the step-by-step process for the final Max Heap.

Given Max Heap (array form): [90, 60, 75, 50, 40, 30, 20, 10, 5, 8]

The standard deletion is **delete-max (remove the root 90)**:

1. **Remove the root** 90 and replace it with the **last element 8**; reduce the size by 1. Array: [8, 60, 75, 50, 40, 30, 20, 10, 5]
2. **Heapify down (sift down):** compare 8 with its children 60 and 75. The larger is 75, so swap. Array: [75, 60, 8, 50, 40, 30, 20, 10, 5]
3. 8 is now at index 2 with children 30 and 20. The larger is 30, so swap. Array: [75, 60, 30, 50, 40, 8, 20, 10, 5]
4. 8 has no children now; stop.

Final Max Heap:



Every parent is greater than or equal to its children. Deleted element: **90**. Time = O(log n).

Q8. Explain the basic operations of a Binary Heap with their time complexities and discuss the applications of priority queues.

A **binary heap** is a complete binary tree stored in an array that satisfies the heap property (max-heap: parent ≥ children; min-heap: parent ≤ children). For index i (0-based): parent = (i−1)/2, left child = 2i+1, right child = 2i+2.

Operation	Description	Time
Insert	Add at the end, sift up	O(log n)
Delete-max / Delete-min (extract)	Replace root with last element, sift down	O(log n)
Get-max / Get-min (peek)	Return the root	O(1)
Heapify	Restore the heap property at a node	O(log n)
Build-heap	Heapify all non-leaf nodes bottom-up	O(n)
Increase/Decrease key	Change the key and sift up/down	O(log n)
Heap sort	Build-heap + n extractions	O(n log n)

Applications of priority queues (implemented with heaps):

1. **Dijkstra's** shortest path and **Prim's** MST algorithms.
2. **Huffman coding** (data compression).
3. **Heap sort.**
4. **CPU/process scheduling** in operating systems.
5. Event-driven simulation and load balancing.
6. Finding the **k-th largest/smallest** element; median maintenance.
7. A* search in AI and network routing/bandwidth management.

UNIT III — Divide and Conquer, Greedy Method

Q1(A). Explain the general methodology of the Divide and Conquer strategy. Illustrate its three fundamental steps with suitable examples.

Divide and Conquer (D&C) solves a problem by breaking it into smaller sub-problems of the same type, solving them recursively, and combining their results.

Three steps:

1. **Divide:** Split the problem of size n into a sub-problems, each of size about n/b.
2. **Conquer:** Solve each sub-problem recursively (a small sub-problem is solved directly — the base case).
3. **Combine:** Merge the sub-solutions into the solution of the original problem.

General recurrence: $T(n) = a \cdot T(n/b) + f(n)$, where f(n) is the cost of dividing and combining. (Master theorem gives the solution.)

General algorithm:

```
DAndC(P):
    if small(P) return Solve(P)
    divide P into P1, P2, ..., Pk
    return Combine( DAndC(P1), DAndC(P2), ..., DAndC(Pk) )
```

Examples:

Problem	Divide	Conquer	Combine	Time
Binary search	Compare with mid element; keep one half	Search that half	Nothing	O(log n)
Merge sort	Split array into two halves	Sort each half	Merge two sorted halves	O(n log n)
Quick sort	Partition around pivot	Sort both partitions	Nothing (in place)	O(n log n) avg

Q1(B). Apply the Binary Search algorithm to locate the desired key in a sorted array. Show low, high and mid at every iteration and determine the total number of comparisons.

Own dataset (15 sorted keys), indices 0 to 14:

Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Key	5	12	18	23	27	31	36	42	49	54	60	67	73	81	90

Search key = 81. mid = ⌊(low + high) / 2⌋

Iteration	low	high	mid	A[mid]	Comparison	Action
1	0	14	7	42	81 > 42	low = mid + 1 = 8
2	8	14	11	67	81 > 67	low = mid + 1 = 12
3	12	14	13	81	81 = 81	**Found at index 13**

Total comparisons = 3 (key found at position 14 in 1-based numbering).

Complexity: Best case O(1) (key at the first mid); worst and average case O(log n). For n = 15 at most $\lfloor \log_2 15 \rfloor + 1 =$ **4 comparisons** are ever required. Prerequisite: the array must be sorted.

Q2. Demonstrate the Merge Sort algorithm on the dataset 42, 17, 8, 33, 25, 61, 14, 50, 9, 37. Draw the splitting and merging stages. Explain the recurrence relation and derive the time complexity.

Splitting (divide) stages:

```
                [42 17 8 33 25 | 61 14 50 9 37]
              /                                  \
      [42 17 8 | 33 25]                    [61 14 50 | 9 37]
       /          \                          /          \
   [42 17 | 8]   [33 | 25]            [61 14 | 50]     [9 | 37]
    /     \                            /     \
 [42]   [17]                        [61]    [14]
```

Merging (combine) stages:

```
[42] + [17]       -> [17 42]            [61] + [14]      -> [14 61]
[17 42] + [8]     -> [8 17 42]          [14 61] + [50]   -> [14 50 61]
[33] + [25]       -> [25 33]            [9] + [37]       -> [9 37]
[8 17 42] + [25 33] -> [8 17 25 33 42]   [14 50 61] + [9 37] -> [9 14 37 50 61]

[8 17 25 33 42] + [9 14 37 50 61]
        -> [8, 9, 14, 17, 25, 33, 37, 42, 50, 61]
```

Sorted output: 8, 9, 14, 17, 25, 33, 37, 42, 50, 61

Recurrence relation:

- Divide: O(1). Conquer: two sub-problems of size n/2. Combine (merge): O(n).
- **T(n) = 2T(n/2) + n**, with T(1) = 1.

Derivation (expansion):

```
T(n) = 2T(n/2) + n
     = 4T(n/4) + 2n
     = 8T(n/8) + 3n
     ...
     = 2^k T(n/2^k) + k·n
```

Put $n/2^k = 1$, so $k = \log_2 n$:

$T(n) = n \cdot T(1) + n \cdot \log_2 n =$ **$\Theta(n \log n)$**

(Master theorem: a = 2, b = 2, $f(n) = n = n^{\log_b a}$, case 2, so Θ(n log n).) Time is O(n log n) in best, average and worst case; extra space is O(n).

Q3. Analyze the working of Quick Sort on 35, 18, 42, 7, 29, 51, 13, 24, 46, taking the first element as the pivot in every step.

Partitioning method: pivot = first element; pointer *i* moves right to find an element > pivot; pointer *j* moves left to find an element ≤ pivot; swap them while i < j; finally swap the pivot with a[j].

Pass 1 — pivot 35 (array 35 18 42 7 29 51 13 24 46):

- i stops at 42, j stops at 24: swap, giving 35 18 **24** 7 29 51 13 **42** 46.
- i stops at 51, j stops at 13: swap, giving 35 18 24 7 29 **13 51** 42 46.
- i crosses j: swap pivot with a[j], giving **13 18 24 7 29 | 35 | 51 42 46**. Pivot 35 is at position 6.

Pass 2 — left part [13 18 24 7 29], pivot 13: result 7 | **13** | 24 18 29, so 13 is at position 2.

Pass 3 — [24 18 29], pivot 24: result 18 | **24** | 29, so 24 is at position 4. (Single elements 7, 18, 29 are already in place.)

Pass 4 — right part [51 42 46], pivot 51: result 46 42 | **51**, so 51 is at position 9.

Pass 5 — [46 42], pivot 46: result 42 | **46**, so 46 is at position 8.

Pass	Sub-array	Pivot	Pivot final position	Result
1	whole array	35	6	13 18 24 7 29 **35** 51 42 46
2	13 18 24 7 29	13	2	7 **13** 24 18 29
3	24 18 29	24	4	18 **24** 29
4	51 42 46	51	9	46 42 **51**
5	46 42	46	8	42 **46**

Sorted array: 7, 13, 18, 24, 29, 35, 42, 46, 51

Complexity comparison:

Case	When	Recurrence	Time
Best case	Pivot always splits array into two equal halves	T(n) = 2T(n/2) + n	**O(n log n)**
Average case	Random pivot positions	similar	O(n log n)
Worst case	Array already sorted/reverse sorted (pivot is the smallest or largest)	T(n) = T(n−1) + n	**O(n^2)**

Space: O(log n) average (recursion stack), O(n) worst case. Quick sort is in-place but not stable.

Q4. Apply the Greedy Job Sequencing algorithm to maximise total profit (each job takes one unit of time).

Given jobs:

Job	J1	J2	J3	J4	J5	J6	J7
Deadline	2	1	2	1	3	2	3
Profit	100	19	27	25	15	35	40

Greedy rule: sort jobs by profit in decreasing order; place each job in the **latest free slot on or before its deadline**; reject it if no slot is free. Maximum deadline = 3, so there are 3 time slots: [0-1], [1-2], [2-3].

Sorted by profit: J1(100, d2), J7(40, d3), J6(35, d2), J3(27, d2), J4(25, d1), J2(19, d1), J5(15, d3)

Step	Job	Deadline	Slot chosen	Decision
1	J1 (100)	2	slot 2	Accepted
2	J7 (40)	3	slot 3	Accepted
3	J6 (35)	2	slot 2 is taken, so slot 1	Accepted
4	J3 (27)	2	slots 2 and 1 full	Rejected
5	J4 (25)	1	slot 1 full	Rejected
6	J2 (19)	1	slot 1 full	Rejected
7	J5 (15)	3	slots 3, 2, 1 full	Rejected

Schedule:

Slot 1	Slot 2	Slot 3
J6	J1	J7

- **Selected jobs: J6, J1, J7. Execution order: J6, then J1, then J7.**
- **Maximum profit = 35 + 100 + 40 = 175.**

Justification: Always taking the highest-profit job first and delaying it as late as possible keeps earlier slots free for jobs with tight deadlines, so no profitable job is blocked unnecessarily. Time complexity: O(n^2) (or O(n log n) with a disjoint-set structure).

Q5. Analyze the Fractional Knapsack problem using the Greedy approach (knapsack

capacity = 50).

Given items:

Item	Weight	Profit	Profit / Weight
I1	10	60	**6.0**
I2	20	100	**5.0**
I3	30	120	**4.0**
I4	15	75	**5.0**
I5	25	125	**5.0**

Greedy rule: sort items by profit/weight ratio in decreasing order and take as much as possible of each until the knapsack is full. Order: I1 (6), then I2, I4, I5 (all 5), then I3 (4).

Step	Item taken	Fraction	Weight added	Profit added	Remaining capacity
1	I1	full	10	60	40
2	I2	full	20	100	20
3	I4	full	15	75	5
4	I5	5/25 = 0.2	5	0.2 x 125 = 25	0

Maximum profit = 60 + 100 + 75 + 25 = 260. Solution vector: x = (I1 = 1, I2 = 1, I3 = 0, I4 = 1, I5 = 0.2). *(Items I2, I4, I5 have equal ratio 5, so any order among them gives the same total of 260.)*

Why greedy works here: items can be **divided**, so filling the capacity with the highest value-per-unit-weight material first can never be improved by exchanging it for a lower ratio item (greedy-choice property + optimal substructure). This is **not** true for 0/1 knapsack. Time: O(n log n) for sorting.

Q6. Construct a Minimum Spanning Tree of the given weighted undirected graph using Kruskal's and Prim's algorithms.

Edges of the graph: F–D 1, E–D 2, B–C 3, A–B 4, F–E 5, C–D 6, A–F 7, A–E 8, C–E 8, B–F 9, B–E 10, B–D 11.

A. Kruskal's algorithm (sort edges by weight; add an edge if it does not form a cycle):

Step	Edge	Weight	Decision	Reason
1	F–D	1	**Accept**	joins F, D
2	E–D	2	**Accept**	joins E to {F, D}
3	B–C	3	**Accept**	new component {B, C}
4	A–B	4	**Accept**	joins A to {B, C}
5	F–E	5	Reject	forms cycle F–D–E
6	C–D	6	**Accept**	joins {A, B, C} with {D, E, F}
7	A–F	7	Reject	cycle
8	A–E	8	Reject	cycle
9	C–E	8	Reject	cycle
10	B–F	9	Reject	cycle
11	B–E	10	Reject	cycle
12	B–D	11	Reject	cycle

MST edges: F–D (1), E–D (2), B–C (3), A–B (4), C–D (6). **Total cost = 1 + 2 + 3 + 4 + 6 = 16.**

B. Prim's algorithm (start at A; repeatedly add the cheapest edge connecting the tree to a new vertex):

Step	Tree vertices	Cheapest crossing edge	Added
1	{A}	A–B (4)	B
2	{A, B}	B–C (3)	C
3	{A, B, C}	C–D (6)	D
4	{A, B, C, D}	D–F (1)	F
5	{A, B, C, D, F}	D–E (2)	E

Total cost = 4 + 3 + 6 + 1 + 2 = 16 (same MST).

```
Minimum Spanning Tree (cost 16):
  A --4-- B --3-- C --6-- D --1-- F
                          |
                          2
                          E
```

Complexities: Kruskal O(E log E); Prim O(E log V) with a heap (O(V^2) with an array).

Q7. Evaluate the shortest paths from source A to all other vertices using Dijkstra's algorithm. Discuss the limitation with negative edge weights.

Directed edges: A→B 4, A→F 7, B→C 3, B→F 8, B→E 2, C→D 6, F→D 3, F→E 5, E→D 2.

Distance table (iteration by iteration). Vertex chosen = unvisited vertex with the smallest distance.

Iter	Chosen	A	B	C	D	E	F
Init	–	0	∞	∞	∞	∞	∞
1	A	0	4	∞	∞	∞	7
2	B (4)	0	4	7	∞	6	7
3	E (6)	0	4	7	8	6	7
4	C (7)	0	4	7	8	6	7
5	F (7)	0	4	7	8	6	7
6	D (8)	0	4	7	8	6	7

(Iteration 2: B relaxes C = 4+3 = 7, F = 4+8 = 12 (no improvement over 7), E = 4+2 = 6. Iteration 3: E relaxes D = 6+2 = 8. Later relaxations give no improvement, e.g. C→D = 13, F→D = 10.)

Shortest paths from A:

Destination	Distance	Path
B	4	A → B
C	7	A → B → C
E	6	A → B → E
F	7	A → F
D	8	A → B → E → D

Limitation with negative edge weights: Dijkstra's greedy assumption is that once a vertex is selected, its distance is final, because any other path can only be longer. With a **negative edge**, a longer path through unvisited vertices can later become shorter, so a finalised distance may be wrong. With a **negative cycle**, shortest paths are not even defined. Hence Dijkstra gives incorrect results on graphs with negative weights; use **Bellman–Ford** instead. Complexity: O(V^2) with an array, O((V + E) log V) with a min-heap.

UNIT IV — Dynamic Programming

Q1. Explain the general methodology of Dynamic Programming. Discuss optimal substructure and overlapping subproblems. Differentiate DP from Divide and Conquer.

Dynamic Programming (DP) solves a complex problem by breaking it into overlapping sub-problems, solving each sub-problem **only once**, and storing its result (in a table) for reuse. It is mainly used for **optimisation problems**.

Steps of DP:

1. Characterise the structure of an optimal solution.
2. Define the value of an optimal solution recursively (recurrence).
3. Compute values **bottom-up** (tabulation) or top-down with memoisation.
4. Construct the optimal solution from the table.

1. Optimal substructure: an optimal solution to the problem contains optimal solutions to its sub-problems. *Example:* in shortest paths, if the shortest path from A to C goes through B, then its A→B and B→C parts are themselves shortest paths (Floyd–Warshall, Bellman–Ford).

2. Overlapping subproblems: the same sub-problems are solved again and again in a naive recursion. *Example:* Fibonacci: F(5) = F(4) + F(3), and F(4) = F(3) + F(2). F(3) is computed twice. DP stores F(i) in a table and reduces $O(2^n)$ to O(n).

Basis	Dynamic Programming	Divide and Conquer
Sub-problems	Overlapping (same sub-problem appears repeatedly)	Independent (no overlap)
Strategy	Solves each sub-problem once and stores the answer	Solves each sub-problem recursively, again every time
Approach	Bottom-up (or memoised top-down)	Top-down
Storage	Needs a table (extra memory)	No table; uses only recursion stack
Efficiency	Polynomial time by avoiding recomputation	Efficient only when sub-problems don't overlap; otherwise exponential
Used for	Optimisation (knapsack, OBST, TSP, shortest paths)	Sorting, searching, matrix multiplication
Examples	Floyd–Warshall, 0/1 knapsack, LCS	Merge sort, quick sort, binary search

Q2. Apply the Bellman–Ford algorithm to find shortest paths from source S in the given directed weighted graph. Show the distance values after each iteration and check for a negative cycle.

Note: the figure refers to an edge list that was not included in the file, so the edge directions below are assumed (left to right as drawn).

Edges (assumed directions): S→A 6, S→E 2, A→B 5, A→E −4, B→C −2, E→C 1, E→D 4, C→D 3. Vertices: S, A, B, C, D, E (|V| = 6, so |V| − 1 = 5 passes are allowed).

Relaxation rule: if d[u] + w(u, v) < d[v] then d[v] = d[u] + w(u, v).

Iteration 1 (edges processed in the order listed):

- S→A: d[A] = 6. S→E: d[E] = 2.
- A→B: d[B] = 6 + 5 = 11. A→E: 6 − 4 = 2, no change.
- B→C: d[C] = 11 − 2 = 9. E→C: 2 + 1 = 3, so d[C] = 3.
- E→D: d[D] = 2 + 4 = 6. C→D: 3 + 3 = 6, no change.

Iteration	S	A	B	C	D	E
Initial	0	∞	∞	∞	∞	∞
1	0	6	11	3	6	2
2	0	6	11	3	6	2
3 to 5	0	6	11	3	6	2

No value changes in iteration 2, so the algorithm can stop early.

Negative cycle check (|V|-th pass): try to relax every edge once more. No distance decreases, therefore **the graph contains no negative-weight cycle** (all edges go "forward" in the order S, A, B, E, C, D, so no cycle exists at all).

Shortest distances from S: A = 6, B = 11, C = 3, D = 6, E = 2.

Vertex	Distance	Path
A	6	S → A
E	2	S → E (or S → A → E, also 2)
C	3	S → E → C
D	6	S → E → D (or S → E → C → D)
B	11	S → A → B

Complexity: O(V·E). Unlike Dijkstra, it handles negative weights and **detects negative cycles** (if any edge can still be relaxed after V − 1 passes).

Q3. Compute the all-pairs shortest path distances using Floyd–Warshall for the given initial cost matrix.

Initial matrix $D^{(0)}$ (vertices 1, 2, 3, 4):

	1	2	3	4
1	0	4	∞	10
2	∞	0	3	∞
3	∞	∞	0	2
4	∞	∞	∞	0

Recurrence: $D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j])$

k = 1 (via vertex 1): no vertex has an edge into vertex 1 (column 1 is all ∞), so $\mathbf{D^{(1)} = D^{(0)}}$.

k = 2 (via vertex 2): D[1][3] = min(∞, D[1][2] + D[2][3]) = 4 + 3 = **7**.

$D^{(2)}$	1	2	3	4
1	0	4	**7**	10
2	∞	0	3	∞
3	∞	∞	0	2
4	∞	∞	∞	0

k = 3 (via vertex 3): D[1][4] = min(10, D[1][3] + D[3][4]) = min(10, 7 + 2) = **9**; D[2][4] = min(∞, 3 + 2) = **5**.

$D^{(3)}$	1	2	3	4
1	0	4	7	**9**
2	∞	0	3	**5**
3	∞	∞	0	2
4	∞	∞	∞	0

k = 4 (via vertex 4): vertex 4 has no outgoing edges, so there is no change. $\mathbf{D^{(4)} = D^{(3)}}$.

Final shortest path matrix:

	1	2	3	4
1	0	4	7	9
2	∞	0	3	5
3	∞	∞	0	2
4	∞	∞	∞	0

Significance of the intermediate vertex: in iteration k, $D^{(k)}[i][j]$ is the shortest path from i to j using only vertices {1, ..., k} as intermediate vertices. Each iteration tests whether passing through vertex k gives a shorter route than the best known so far. After k = n, all vertices are allowed, so the matrix is final.

Time complexity: three nested loops (k, i, j), each running n times, so each of the n iterations does n^2 comparisons: $\mathbf{T(n) = n \times n^2 = \Theta(n^3)}$. Space: $\Theta(n^2)$.

Q4. Optimal Binary Search Tree: n = 4, keys {do, if, int, while}, p(1:4) = {3, 3, 1, 1}, q(0:4) = {2, 3, 1, 1, 1}.

Formulas:

- $W(i, i) = q_i$; $C(i, i) = 0$; $R(i, i) = 0$.
- $W(i, j) = W(i, j-1) + p_j + q_j$
- $C(i, j) = W(i, j) + \min$ over $i < k \leq j$ of $[\ C(i, k-1) + C(k, j)\ ]$; $R(i, j)$ = the k giving the minimum.

Keys: a_1 = do, a_2 = if, a_3 = int, a_4 = while.

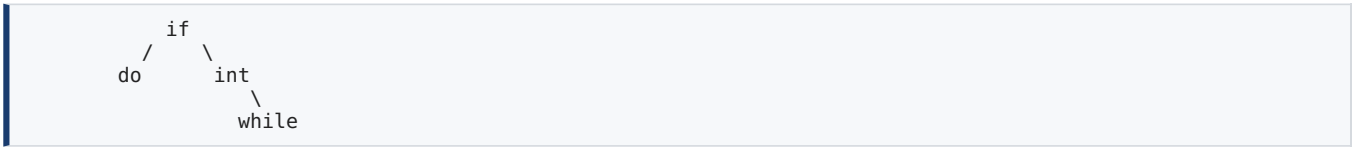
Cost table (W, C, R for each j − i):

j − i	(i, j)	W	C	R (root)
0	(0,0) (1,1) (2,2) (3,3) (4,4)	2, 3, 1, 1, 1	0	0
1	(0,1)	3+3+2 = 8	8	1
1	(1,2)	3+1+3 = 7	7	2
1	(2,3)	1+1+1 = 3	3	3
1	(3,4)	1+1+1 = 3	3	4
2	(0,2)	8+1+3 = 12	12 + min(k=1: 0+7, k=2: 8+0) = 19	1
2	(1,3)	7+1+1 = 9	9 + min(k=2: 0+3, k=3: 7+0) = 12	2
2	(2,4)	3+1+1 = 5	5 + min(k=3: 0+3, k=4: 3+0) = 8	3
3	(0,3)	12+1+1 = 14	14 + min(k=1: 0+12, k=2: 8+3, k=3: 19+0) = 25	2
3	(1,4)	9+1+1 = 11	11 + min(k=2: 0+8, k=3: 7+3, k=4: 12+0) = 19	2
4	(0,4)	14 + 1 + 1 = 16	16 + min(k=1: 0+19, k=2: 8+8, k=3: 19+3, k=4: 25+0) = **32**	**2**

($W(0,4) = W(0,3) + p_4 + q_4 = 14 + 1 + 1 = 16$. For (2,4) the choices k = 3 and k = 4 tie at 8; k = 3 is taken.)

Finding the tree: R(0,4) = 2, so the root is $\mathbf{a_2}$ = **if**. Left subtree = keys (0,1), whose root is R(0,1) = 1, i.e. **do**. Right subtree = keys (2,4), whose root is R(2,4) = 3, i.e. **int**; its right subtree is (3,4) with root R(3,4) = 4, i.e. **while**.

Final Optimal BST:



Minimum cost C(0,4) = 32, i.e. the minimum expected search cost = C / W = 32 / 16 = **2** comparisons per search

(weighted). Time: $O(n^3)$ ($O(n^2)$ with Knuth's optimisation).

Q5. Solve the 0/1 Knapsack problem using Dynamic Programming (capacity = 10).

Item	I1	I2	I3	I4	I5
Weight	2	3	4	5	6
Profit	12	20	25	30	35

Recurrence: V[i][w] = V[i−1][w] if w_i > w, else max(V[i−1][w], p_i + V[i−1][w − w_i]).

DP table V[i][w] (rows = items considered, columns = capacity 0 to 10):

i \ w	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1 (I1)	0	0	12	12	12	12	12	12	12	12	12
2 (I2)	0	0	12	20	20	32	32	32	32	32	32
3 (I3)	0	0	12	20	25	32	37	45	45	57	57
4 (I4)	0	0	12	20	25	32	37	45	50	57	**62**
5 (I5)	0	0	12	20	25	32	37	45	50	57	**62**

Maximum profit = V[5][10] = 62.

Traceback (selected items):

- V[5][10] = V[4][10] = 62, so I5 is not selected.
- V[4][10] = 62 ≠ V[3][10] = 57, so **I4 is selected** (w = 5); remaining capacity = 5.
- V[3][5] = 32 = V[2][5], so I3 is not selected.
- V[2][5] = 32 ≠ V[1][5] = 12, so **I2 is selected** (w = 3); remaining capacity = 2.
- V[1][2] = 12 ≠ V[0][2] = 0, so **I1 is selected** (w = 2); remaining capacity = 0.

Selected items: I1, I2, I4. Total weight = 2 + 3 + 5 = 10, total profit = 12 + 20 + 30 = 62.

Why ratio-based greedy may fail in 0/1 knapsack: profit/weight ratios are I2 = 6.67, I3 = 6.25, I1 = 6.0, I4 = 6.0, I5 = 5.83. Greedy takes I2, I3, I1 (weight 9, profit 57); the remaining capacity 1 cannot hold any other item. Profit = **57 < 62**. Items cannot be split, so the highest-ratio picks can leave unusable capacity, while the DP considers all combinations of items. Time and space: O(n·W).

Q6. Analyze the Travelling Salesperson Problem using Dynamic Programming (Held-Karp), starting and ending at city A.

Cost matrix (A, B, C, D):

	A	B	C	D
A	0	10	15	20
B	10	0	35	25
C	15	35	0	30
D	20	25	30	0

Held-Karp recurrence: g(i, S) = min over j in S of [c(i, j) + g(j, S − {j})], with g(i, ∅) = c(i, A). g(i, S) = the shortest path starting at i, visiting all cities in S exactly once and ending at A.

Step 1: S = ∅: g(B, ∅) = 10, g(C, ∅) = 15, g(D, ∅) = 20.

Step 2: |S| = 1:

- g(B, {C}) = c(B,C) + g(C, ∅) = 35 + 15 = **50**; g(B, {D}) = 25 + 20 = **45**
- g(C, {B}) = 35 + 10 = **45**; g(C, {D}) = 30 + 20 = **50**
- g(D, {B}) = 25 + 10 = **35**; g(D, {C}) = 30 + 15 = **45**

Step 3: |S| = 2:

- g(B, {C, D}) = min(c(B,C) + g(C,{D}), c(B,D) + g(D,{C})) = min(35 + 50, 25 + 45) = min(85, 70) = **70** (via D)
- g(C, {B, D}) = min(c(C,B) + g(B,{D}), c(C,D) + g(D,{B})) = min(35 + 45, 30 + 35) = min(80, 65) = **65** (via D)
- g(D, {B, C}) = min(c(D,B) + g(B,{C}), c(D,C) + g(C,{B})) = min(25 + 50, 30 + 45) = min(75, 75) = **75**

Step 4: final: g(A, {B, C, D}) = min(c(A,B) + g(B,{C,D}), c(A,C) + g(C,{B,D}), c(A,D) + g(D,{B,C})) = min(10 + 70, 15 + 65, 20 + 75) = min(**80**, **80**, 95) = **80**

Optimal tour: A → B → D → C → A, cost = 10 + 25 + 30 + 15 = **80**. (Its reverse A → C → D → B → A also costs 80.)

Use of overlapping subproblems: every g(i, S) is computed **once**, stored in the table and then reused by all larger subsets that contain it. For example, the stored values g(B,{C,D}) = 70, g(C,{B,D}) = 65 and g(D,{B,C}) = 75 are all reused when computing g(A,{B,C,D}), instead of re-enumerating the underlying paths. With only 3 cities the sharing is small, but for n cities a value g(j, S) is shared by many different longer paths, which is why DP needs $O(n^2 \cdot 2^n)$ time instead of checking all $(n-1)!$ tours (here 3! = 6). Space: $O(n \cdot 2^n)$.

UNIT V — Backtracking and Branch & Bound

Q1. Explain the general methodology of Backtracking. Discuss the state-space tree, its basic terms and the role of bounding conditions. Distinguish it from exhaustive search.

Backtracking builds a solution one component at a time (a depth-first search of the solution space). After each choice it checks whether the partial solution can still lead to a complete valid solution; if not, it **abandons (prunes)** that branch and **backtracks** to try another choice.

State-space tree: a tree whose nodes represent partial solutions. The root = empty solution; each level represents a choice for one component; the leaves are complete solutions (or dead ends).

Basic terms:

Term	Meaning
Solution space	All possible tuples (x_1, ..., x_n)
Explicit constraints	Rules restricting each x_i to a set of values (e.g. $x_i \in \{0, 1\}$)
Implicit constraints	Rules describing how the x_i must relate to one another (e.g. no two queens attack)
Live node	A generated node whose children have not all been generated
E-node (expanding node)	The live node currently being expanded
Dead node	A node that is not expanded further (it is infeasible, or all children are done)
Promising node	A node that may still lead to a solution
Non-promising node	A node that cannot lead to a solution; it is pruned

Role of bounding conditions (promising function): a bounding function is checked at every node to decide whether it can lead to a solution. If it returns false, the whole subtree is **pruned** without being explored, which saves enormous effort.

Generic algorithm:

```
Backtrack(k):
    for each value v in the choices for x[k]:
        x[k] = v
        if Bound(x[1..k]) is true:       // promising
            if x[1..k] is a complete solution: output it
            else Backtrack(k + 1)
```

Example (4-Queens): place queens row by row; when a queen in a row has no safe column, return to the previous row and move that queen.

Backtracking	Exhaustive search (brute force)
Builds solutions incrementally and **prunes** invalid partial solutions	Generates **all** possible solutions, then tests each
Uses bounding/promising functions	No pruning
Explores only a fraction of the space	Explores the entire space
Much faster in practice (worst case still exponential)	Always the full exponential/factorial cost
Example: 4-Queens checks about 26 nodes	$4^4 = 256$ placements (or $4! = 24$ permutations)

Q2. Apply Backtracking to the 4-Queens problem. Draw the state-space tree and show the solutions.

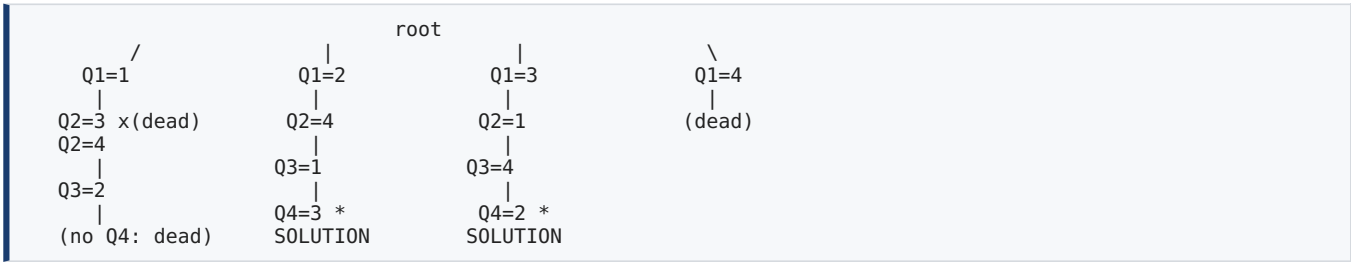
Problem: place 4 queens on a 4 x 4 board so that no two share a row, column or diagonal. Place one queen per row. Let x_i = column of the queen in row i.

Promising (safe) test: queen k at column x_k is safe if for every earlier queen $i < k$: $x_i \neq x_k$ and $|x_i - x_k| \neq |i - k|$.

Trace of the search (row, column):

- **Q1 = (1,1).**
  - Row 2: col 1 unsafe (column), col 2 unsafe (diagonal), **col 3 safe** → Q2 = (2,3).
    - Row 3: col 1 (column), col 2 (diagonal with Q2), col 3 (column), col 4 (diagonal with Q2): **all unsafe → backtrack.**
  - Row 2: **col 4 safe** → Q2 = (2,4).
    - Row 3: col 1 unsafe (column), **col 2 safe** → Q3 = (3,2).
      - Row 4: col 1 (column), col 2 (column), col 3 (diagonal with Q3), col 4 (column): **all unsafe → backtrack.**
    - Row 3: col 3 unsafe (diagonal with Q2), col 4 unsafe (column) → **backtrack to row 1.**
- **Q1 = (1,2).**
  - Row 2: col 1 (diagonal), col 2 (column), col 3 (diagonal) unsafe; **col 4 safe** → Q2 = (2,4).
    - Row 3: **col 1 safe** → Q3 = (3,1).
      - Row 4: col 1, col 2 (columns), col 4 (column) unsafe; **col 3 safe** → Q4 = (4,3). **Solution 1 found: (2, 4, 1, 3).**
- **Q1 = (1,3):** by symmetry, finds **Solution 2: (3, 1, 4, 2)**.
- **Q1 = (1,4):** mirror image of Q1 = (1,1); no solution.

State-space tree (partial):



The two distinct solutions (Q = queen):

```
Solution 1: (2,4,1,3)      Solution 2: (3,1,4,2)
 . Q . .                    . . Q .
 . . . Q                    Q . . .
 Q . . .                    . . . Q
 . . Q .                    . Q . .
```

How the promising function eliminates invalid configurations: as soon as a queen is placed in the same column or on the same diagonal as an earlier queen, that branch is cut off and none of its descendants is generated. This avoids examining the $4^4 = 256$ (or $4! = 24$) complete placements; only about 26 nodes of the state-space tree are generated.

Q3. Use Backtracking to find all subsets of S = {5, 10, 12, 13, 15, 18} whose sum is 30. Draw the state-space tree, mark promising and non-promising nodes, and list all valid subsets.

(The set was written as "51012131518" in the question paper, read here as 5, 10, 12, 13, 15, 18, which is already in ascending order.)

Setup: M = 30, items in ascending order. Total = 73. Each level decides whether to **include (1)** or **exclude (0)** the next element. For a node with current sum **s** and sum of remaining elements **r**:

Pruning (bounding) conditions — a node is promising only if:

1. **$s + w_{next} \le M$** (to include the next item) — otherwise adding exceeds 30 (prune).
2. **$s + r \ge M$** — otherwise even adding all remaining items cannot reach 30 (prune).
3. If **$s = M$**, a solution is recorded.

State-space tree (include branch first). Each node shown as (s, r):

```
(0,73)
 |-- include 5 --> (5,68)
 |      |-- include 10 --> (15,58)
 |      |      |-- include 12 --> (27,46)
 |      |      |      |-- include 13 --> 40 > 30   X pruned
 |      |      |      '-- exclude 13 --> (27,33): 15 -> 42, 18 -> 45 >30   X (no solution)
 |      |      '-- exclude 12 --> (15,46)
 |      |             |-- include 13 --> (28,33): add 15 or 18 exceeds 30   X
 |      |             '-- exclude 13 --> (15,33)
 |      |                    |-- include 15 --> (30,18)   *** SOLUTION {5,10,15} ***
 |      |                    '-- exclude 15 --> (15,18): +18 = 33 > 30       X
 |      '-- exclude 10 --> (5,58)
 |             |-- include 12 --> (17,46)
 |             |      |-- include 13 --> (30,33)   *** SOLUTION {5,12,13} ***
 |             |      '-- exclude 13 --> 17+15 = 32 > 30, 17+18 = 35 > 30    X
 |             '-- exclude 12 --> (5,46): 13, 15, 18 combinations give 18, 20, 23, 5+... < 30 or > 30   X
 '-- exclude 5 --> (0,68)
        |-- include 10 --> (10,58): every extension is 22, 25, 23, 28, 35+ ...   X (no 30)
        '-- exclude 10 --> (0,58)
               |-- include 12 --> (12,46)
               |      |-- include 13 --> (25,33)   X (25 + 15 = 40 > 30)
               |      '-- exclude 13 --> (12,33): include 15 -> 27 (no); exclude 15 -> include 18 -> (30,0)
               |                                                      *** SOLUTION {12,18} ***
               '-- exclude 12 --> (0,46): 13+15 = 28, 13+18 = 31, 15+18 = 33, 13+15+18 = 46  X
```

Examples of non-promising (pruned) nodes:

- (5+10+12+13) = 40 > 30 fails condition 1 (sum exceeds M).
- (s = 5, excluding 10, 12, 13 and 15, r = 18): 5 + 18 = 23 < 30 fails condition 2.
- (s = 10, excluding 12, 13, 15, r = 18): 10 + 18 = 28 < 30 fails condition 2.
- (s = 0, excluding 10, 12, 13, 15, r = 18): 18 < 30 fails condition 2.

All valid subsets with sum 30:

Subset	Check
{5, 10, 15}	5 + 10 + 15 = 30
{5, 12, 13}	5 + 12 + 13 = 30
{12, 18}	12 + 18 = 30

Solution vectors (x1…x6 for 5,10,12,13,15,18): (1,1,0,0,1,0), (1,0,1,1,0,0), (0,0,1,0,0,1).

The two pruning rules cut the $2^6 = 64$ leaf combinations down to a small part of the tree.

Q4. Graph colouring with at most 3 colours using Backtracking (given undirected graph).

The given graph has vertices A, B, C, D, E, and every pair of vertices is joined by an edge (all 10 edges: AB, AC, AD, AE, BC, BD, BE, CD, CE, DE), so it is the **complete graph K_5**.

Colouring rule: adjacent vertices must have different colours. Colours available: {1, 2, 3}. **Promising test:** vertex k can take colour c only if none of its already-coloured neighbours has colour c.

State-space tree (order A, B, C, D, E):

```
A = 1   (by symmetry, any colour for A gives the same outcome; 3 choices)
 |-- B = 1 : X (adjacent to A)
 |-- B = 2
 |      |-- C = 1 : X (A)      C = 2 : X (B)
 |      |-- C = 3
 |             |-- D = 1 : X (A)   D = 2 : X (B)   D = 3 : X (C)
 |             '-- no colour available for D  -> BACKTRACK
 '-- B = 3
        |-- C = 2
               '-- D: 1 X (A), 2 X (C), 3 X (B)  -> BACKTRACK
(similarly for the other branches: every branch fails at vertex D)
```

- The first three vertices A, B, C are mutually adjacent, so they must use **three different colours** (3! = 6 ways).
- Vertex D is adjacent to A, B and C, so all three colours are already used by its neighbours: **no colour is available** → backtrack to the previous levels, try all alternatives, and every path fails at D.
- Vertex E would also need a 5th distinct colour.

Number of valid 3-colourings = 0. The graph is **not 3-colourable**. Its chromatic number is **5** (a complete graph on n vertices needs n colours). With 5 colours there would be 5! = 120 valid colourings.

Time complexity: $O(m^n)$ in the worst case for m colours and n vertices; pruning stops the search early.

Q5. Apply Backtracking to find all Hamiltonian cycles in the given undirected graph, starting from vertex A.

Edges of the graph: A–B, A–C, A–E, B–C, B–D, B–E, C–E, D–E (8 edges; there is no A–D and no C–D edge). Adjacency: A: B, C, E; B: A, C, D, E; C: A, B, E; D: B, E; E: A, B, C, D.

A **Hamiltonian cycle** visits every vertex exactly once and returns to the start. At level k, vertex x_k is promising only if (i) it is adjacent to x_{k-1} and (ii) it is not already in the path; at the last level it must also be adjacent to the start vertex A.

State-space tree (paths from A):

```
A
|-- B
|    |-- C
|    |    '-- E
|    |         '-- D  --> D is not adjacent to A        X (cannot close the cycle)
|    |-- D
|    |    '-- E
|    |         '-- C  --> C adjacent to A                *** A-B-D-E-C-A ***
|    '-- E
|         |-- C  --> D left, but C-D is not an edge     X (dead end)
|         '-- D  --> C left, but D-C is not an edge     X (dead end)
|-- C
|    |-- B
|    |    |-- D
|    |    |    '-- E  --> E adjacent to A               *** A-C-B-D-E-A ***
|    |    '-- E
|    |         '-- D  --> D not adjacent to A           X
|    '-- E
|         |-- D
|         |    '-- B  --> B adjacent to A               *** A-C-E-D-B-A ***
|         '-- B
|              '-- D  --> D not adjacent to A           X
'-- E
     |-- B
     |    |-- C --> D unreachable from C                X
     |    '-- D --> C unreachable from D                X
     |-- C
     |    '-- B
     |         '-- D --> D not adjacent to A            X
     '-- D
          '-- B
               '-- C --> C adjacent to A                *** A-E-D-B-C-A ***
```

How invalid paths are rejected: a partial path is abandoned when the next vertex is not adjacent to the current one, when it was already visited, or when the last vertex has no edge back to A. Vertex D has only two neighbours (B and E), so any Hamiltonian cycle must use both edges B–D and D–E; every path that visits D in another way is a dead end.

Cycles found (4 directed paths): A-B-D-E-C-A, A-C-B-D-E-A, A-C-E-D-B-A, A-E-D-B-C-A.

Treating reverse traversals as the same cycle: A-B-D-E-C-A is the reverse of A-C-E-D-B-A, and A-C-B-D-E-A is the reverse of A-E-D-B-C-A.

Distinct Hamiltonian cycles = 2:

1. **A – B – D – E – C – A**
2. **A – C – B – D – E – A**

Q6. Analyze the 0/1 Knapsack problem using Least Cost (LC) Branch and Bound (capacity = 16).

Item	I1	I2	I3	I4	I5
Weight	2	5	10	5	4
Profit	40	30	50	10	25
Profit / Weight	20	6	5	2	6.25

Method: arrange items in decreasing profit/weight order: **I1 (2, 40), I5 (4, 25), I2 (5, 30), I3 (10, 50), I4 (5, 10)**. Each node decides *include* or *exclude* the next item. For a node with weight W and profit P:

- **Upper bound UB** = P + profit of the following items taken greedily, the last one taken **fractionally** to fill the capacity. (This is the optimistic estimate.)
- **Cost c(x) = −UB** (the algorithm minimises cost, so a larger profit means a more negative cost), and the best **feasible profit found so far** acts as the lower limit U.
- **Pruning rule:** kill a node if its UB ≤ the best feasible profit already found (equivalently c(x) > U). The live node with the **least cost (largest UB)** is expanded next.

State-space tree (node: weight, profit, UB):

Node	Decision	Weight	Profit	UB	Status
Root	–	0	0	40 + 25 + 30 + 5·5 = **120**	expand
1	include I1	2	40	120	expand (least cost)
2	exclude I1	0	0	25 + 30 + (7/10) x 50 = **90**	**pruned** later (UB 90 < 115)
3	I1, include I5	6	65	120	expand
4	I1, exclude I5	2	40	115	**pruned** (UB ≤ 115)
5	I1, I5, include I2	11	95	120	expand
6	I1, I5, exclude I2	6	65	115	expand, leads to the best solution
7	I1, I5, I2, include I3	21 > 16	–	–	**infeasible** (killed)
8	I1, I5, I2, exclude I3	11	95	105	feasible: add I4 (w 16) gives profit 105, so **best so far = 105**; UB 105 ≤ 115 → pruned later
9	I1, I5, exclude I2, **include I3**	**16**	**115**	**115**	**feasible, best = 115**
10	I1, I5, exclude I2, exclude I3	6	65	75	**pruned** (UB 75 < 115)

Once a feasible solution of profit **115** is found (node 9), every live node with UB ≤ 115 is killed (nodes 2, 4, 8, 10). No live node remains, so the search ends.

Optimal solution: items I1, I5 and I3 (x = (1, 0, 1, 0, 1) for I1…I5). Weight = 2 + 4 + 10 = **16** (full), **maximum profit = 40 + 25 + 50 = 115.**

(Branch and Bound avoided exploring the full tree of $2^5 = 32$ leaves; only about 10 nodes were generated.)

Q7. Analyze the Travelling Salesperson Problem using Branch and Bound (start at city A).

Cost matrix (A, B, C, D, E):

	A	B	C	D	E
A	∞	12	10	19	8
B	12	∞	3	7	6
C	10	3	∞	2	20
D	19	7	2	∞	4
E	8	6	20	4	∞

Method (reduced cost matrix):

1. **Root lower bound:** subtract the minimum of every row, then of every column; the total subtracted is the lower bound.
   - Row minima: 8 (A), 3 (B), 2 (C), 2 (D), 4 (E) → sum = 19.
   - After row reduction, column minima: A = 0, B = 1, C = 0, D = 0, E = 0 → sum = 5. (Column B minimum is 1.)
   - **Lower bound of root = 19 + 5 = 24.**
2. **For a child node (edge i → j):** cost(child) = cost(parent) + M[i][j] + reduction cost of the new matrix, where row i and column j are set to ∞ and the entry M[j][A] is set to ∞ (to prevent early return to A); then the matrix is reduced again.
3. Always expand the live node with the **lowest bound**; prune any node whose bound ≥ the best complete tour found.

Reduced matrix at root (bound 24):

	A	B	C	D	E
A	∞	3	2	11	0
B	5	∞	0	4	3
C	4	0	∞	0	18
D	13	4	0	∞	2
E	0	1	16	0	∞

State-space tree with lower bounds:

Level	Node (path)	Computation	Lower bound	Status
0	A (root)	19 + 5	**24**	expand
1	A→B	24 + 3 + 2	29	live
1	A→C	24 + 2 + 5	31	live
1	A→D	24 + 11 + 2	37	live
1	**A→E**	24 + 0 + 4	**28**	**expand (lowest)**
2	A→E→B	28 + 1 + 0	29	live
2	A→E→C	28 + 16 + 5	49	pruned
2	**A→E→D**	28 + 0 + 0	**28**	**expand (lowest)**
3	A→E→D→B	28 + 4 + 0	32	live
3	**A→E→D→C**	28 + 0 + 1	**29**	**expand**
4	A→E→D→C→B	29 + 0 + 0	**29**	**complete tour, A→E→D→C→B→A with cost 29**

After the tour of cost **29** is found, all remaining live nodes with bound ≥ 29 (A→C = 31, A→D = 37, A→E→C = 49, A→E→D→B = 32, and A→B, A→E→B at 29 which cannot improve) are **pruned**.

Optimal tour: A → E → D → C → B → A Cost = 8 + 4 + 2 + 3 + 12 = **29**. *(The reverse tour A → B → C → D → E → A has the same cost 29.)*

Complexity: worst case O(n!) but pruning makes it far faster in practice; bounds come from matrix reduction ($O(n^2)$ per node).

Q8. Compare and contrast Backtracking and Branch and Bound.

Aspect	Backtracking	Branch and Bound
Used for	Decision and enumeration problems (find all or any feasible solution)	**Optimisation** problems (find the best solution: minimum cost or maximum profit)
Tree traversal	Depth-First Search of the state-space tree	BFS, least-cost (best-first) or DFS (FIFO/LIFO/priority queue)
Pruning basis	Feasibility: constraints are violated (bounding/promising function)	Bound: the node's best possible value cannot beat the best solution found so far
Bounding function	Checks only constraint satisfaction	Computes a lower/upper bound of the objective function
Solution	May generate **all** solutions	Finds an **optimal** solution (only the best)
Node selection	Next child of the current node (stack/recursion)	Live node with the best bound (priority queue)
Memory	Low (recursion stack, O(depth))	Higher (stores many live nodes)
Speed	Fast for constraint problems	Faster than backtracking for optimisation
Examples	N-Queens, Sum of subsets, Graph colouring, Hamiltonian cycle	0/1 Knapsack, TSP, Job assignment
Worst case	Exponential	Exponential

Similarities: both explore a state-space tree, both prune non-promising nodes, and both avoid the brute-force enumeration of all possibilities.

Important Questions for Revision

A. Most Expected (from your question paper, practise these first)

Unit I

1. Describe the asymptotic notations O, Ω and Θ with examples.
2. Explain collision resolution techniques (linear probing, quadratic probing, double hashing, separate chaining).
3. Differentiate static and dynamic hashing; explain extendable hashing with an example.

Unit II

1. Construct an AVL tree for the given keys, showing all rotations.
2. Show deletion of a node in an AVL tree with balancing.
3. Construct a B-tree of order 4 for the given keys.
4. Build a max-heap/min-heap by insertion and perform delete-max.

Unit III

1. Merge sort on the given data with the recurrence and time complexity.
2. Quick sort trace with best-case and worst-case complexity.
3. Job sequencing with deadlines and fractional knapsack.
4. Kruskal's and Prim's MST; Dijkstra's shortest path.

Unit IV

1. Optimal Binary Search Tree (cost table, root table, final tree).
2. 0/1 Knapsack using DP table and traceback.
3. Floyd–Warshall matrices and complexity; Bellman–Ford iterations.
4. TSP using Held–Karp.

Unit V

1. 4-Queens state-space tree; sum of subsets; Hamiltonian cycle; graph colouring.
2. 0/1 Knapsack and TSP using Branch and Bound.
3. Backtracking vs Branch and Bound.

B. Extra Practice Questions (may also be asked)

1. State the Master theorem and solve $T(n) = 4T(n/2) + n^2$.
2. What is a Red-Black tree? List its properties and compare it with an AVL tree.
3. Write the heap sort algorithm and trace it on a sample array.
4. Explain Huffman coding with an example (greedy method).
5. Explain topological sorting and its applications.
6. Find the Longest Common Subsequence (LCS) of two strings using DP.
7. Explain Matrix Chain Multiplication using DP with an example.
8. Explain the Strassen's matrix multiplication algorithm and its complexity.
9. Differentiate between P, NP, NP-hard and NP-complete problems.
10. What is a B+ tree? How does it differ from a B-tree?
11. Explain linear hashing and rehashing; what is the load factor?
12. Write the algorithm for the N-Queens problem and calculate its complexity.
13. Solve the Assignment problem or 15-puzzle using Branch and Bound.
14. Compare Kruskal's and Prim's algorithms.

Quick Revision Tips

1. **Unit I:** Learn the three asymptotic definitions with constants, the hash functions and the 4 collision techniques.
2. **Unit II:** Practise AVL rotations (LL, RR, LR, RL), B-tree split/merge and heap sift-up/sift-down.
3. **Unit III:** Remember the recurrences $T(n) = 2T(n/2) + n$ (merge sort), the greedy sorting rule for each problem and the MST/Dijkstra tables.
4. **Unit IV:** Memorise the DP recurrences (Floyd, OBST, 0/1 knapsack, Held–Karp) and write tables neatly.
5. **Unit V:** State-space trees with bounding conditions; know the difference between Backtracking and Branch and Bound.

All the best for your exam!